A. Soal Essay

1. Apa itu Normalisasi Database? Normalisasi database itu intinya adalah proses merapikan struktur *table* dan *column* di dalam *database*. Tujuannya ada dua:

- **Mengurangi redundansi:** Biar nggak ada data yang berulang-ulang atau diduplikat tanpa alasan yang jelas.
- **Menjaga *data integrity* (integritas data):** Biar kalau kita mau nambahin (*insert*), ngedit (*update*), atau ngapus (*delete*) data, nggak terjadi *error* atau anomali (misalnya ada data yang ikut kehapus padahal nggak seharusnya).

2. Kapan database perlu di-normalisasi dan kapan tidak perlu?

- **Kapan perlu di-normalisasi?** Sangat perlu waktu kita bikin sistem yang transaksional atau OLTP (*Online Transaction Processing*). Contohnya aplikasi kasir, *e-commerce*, atau sistem absensi karyawan. Di sistem kayak gini, proses *insert, update,* dan *delete* data terjadi sangat sering dan cepat. Normalisasi bikin *database* lebih enteng buat nulis data baru dan mastiin datanya selalu konsisten.
- **Kapan tidak perlu di-normalisasi (Denormalisasi)?** Nggak perlu atau malah disarankan buat di-denormalisasi kalau kita bikin *Data Warehouse* atau sistem OLAP (*Online Analytical Processing*). Sistem ini fokusnya buat *reporting* atau analisis data sejarah. Kalau datanya terlalu di-normalisasi, kita bakal butuh banyak banget proses *JOIN table* pas ngejalanin *query*. Akibatnya, *loading report* atau *dashboard* bakal jadi lambat banget. Jadi, di sini kecepatan baca data (*read speed*) lebih penting daripada ngehemat *storage*.

B. Soal Case

Dari deskripsi studi kasus perusahaan pengolahan hasil laut, kita bisa mecah datanya jadi beberapa *Entity* dan *Relationship* untuk ERD, lalu nge-*map* ke *Relational Model*.

1. Rancangan Entity-Relationship Diagram (ERD)

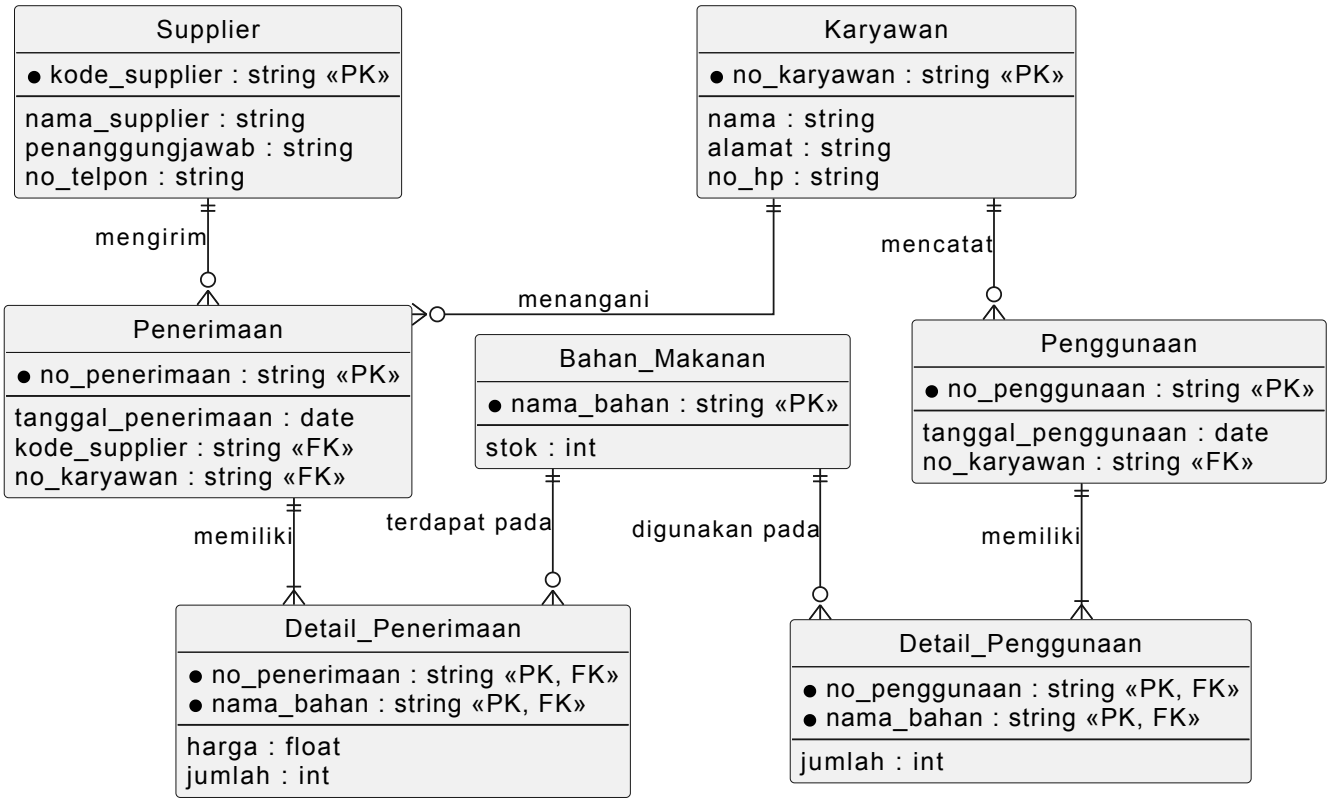
A. Entities & Attributes:

- **Karyawan:** `no_karyawan` (PK), `nama`, `alamat`, `no_hp`
- **Supplier:** `kode_supplier` (PK), `nama_supplier`, `penanggungjawab`, `no_telpon`
- **Bahan Makanan:** `nama_bahan` (PK) *(Note: Soal minta nama yang unik, jadi kita jadikan Primary Key)*, `stok`
- **Penerimaan:** `no_penerimaan` (PK), `tanggal_penerimaan`
- **Penggunaan:** `no_penggunaan` (PK) *(Kita tambahkan ID ini biar gampang nyatet log tiap kali ada pemakaian)*, `tanggal_penggunaan`

B. Relationships (Kardinalitas):

- **Supplier (1) ke Penerimaan (M):** Satu *supplier* bisa ngirim banyak penerimaan, tapi satu nomor penerimaan pasti cuma dari satu *supplier*.
- **Karyawan (1) ke Penerimaan (M):** Satu karyawan bisa nanganin banyak penerimaan (pengiriman dari *supplier*).

- **Karyawan (1) ke Penggunaan (M):** Satu karyawan bisa nyatet banyak data penggunaan bahan.
- **Penerimaan (M) ke Bahan Makanan (M):** Satu nota penerimaan bisa berisi banyak bahan, dan satu bahan bisa ada di banyak nota penerimaan. *(Ini relasi Many-to-Many, harus dipecah pakai tabel detail di relational model nanti, dengan atribut tambahan: `harga` dan `jumlah`).*
- **Penggunaan (M) ke Bahan Makanan (M):** Satu log penggunaan bisa ngambil banyak bahan, dan satu bahan bisa dipakai di banyak log penggunaan. *(Ini juga Many-to-Many, dengan atribut tambahan: `jumlah`).*



2. Mapping ke Relational Database (Relational Model)

*(Keterangan: **Bold** = Primary Key, Italic = Foreign Key)*

1. **Karyawan** (**no_karyawan**, nama, alamat, no_hp)
2. **Supplier** (**kode_supplier**, nama_supplier, penanggungjawab, no_telpon)
3. **Bahan_Makanan** (**nama_bahan**, stok)
4. **Penerimaan** (**no_penerimaan**, tanggal_penerimaan, *kode_supplier*, *no_karyawan*)
5. **Detail_Penerimaan** (***no_penerimaan***, ***nama_bahan***, harga, jumlah)

- *Note: `no_penerimaan` dan `nama_bahan` gabung jadi satu Composite Primary Key.*

6. **Penggunaan** (**no_penggunaan**, tanggal_penggunaan, *no_karyawan*)
7. **Detail_Penggunaan** (***no_penggunaan***, ***nama_bahan***, jumlah)

- *Note: `no_penggunaan` dan `nama_bahan` gabung jadi satu Composite Primary Key.*